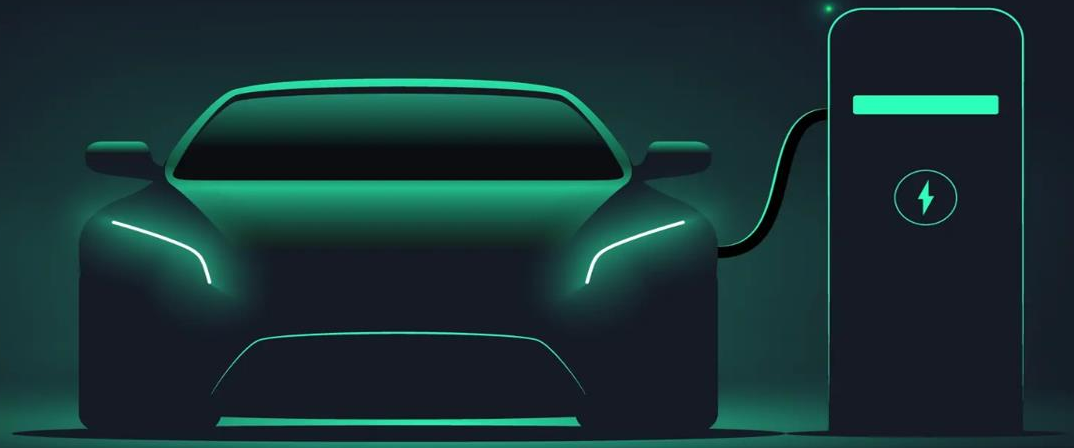
EV Market Analysis



Codebasics Resume Challenge #12 | AtliQ Motors India Expansion

Q1 Top & Bottom EV Makers — 2-Wheelers (FY2023 & FY2024)

Objective

Identify top 3 and bottom 3 electric 2-wheeler manufacturers by total units sold for each fiscal year. Helps AtliQ understand the competitive landscape.

SQL Concepts Used

- Stored Procedure
- RANK() OVER()
- PARTITION BY
- CASE WHEN / IF
- JOIN + USING

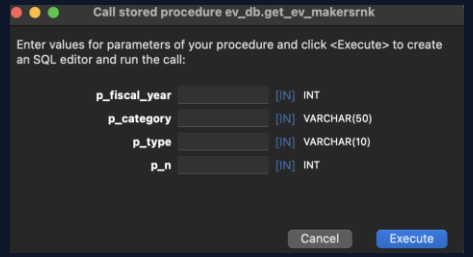
Output Columns

- fiscal_year
- maker
- total_ev_sold
- rnk (rank)

SQL

```sql
CREATE DEFINER=`root`@`localhost` PROCEDURE `get_ev_makersrnk`(
    IN p_fiscal_year INT,
    IN p_category VARCHAR(50),
    IN p_type VARCHAR(10),
    IN p_n INT
)
BEGIN
    SELECT *
    FROM (
        SELECT
            fiscal_year,
            maker,
            SUM(electric_vehicles_sold) AS total_ev_sold,
            RANK() OVER (
                PARTITION BY fiscal_year
                ORDER BY
                    CASE WHEN p_type = 'TOP' THEN SUM(electric_vehicles_sold) END DESC,
                    CASE WHEN p_type = 'BOTTOM' THEN SUM(electric_vehicles_sold) END ASC
            ) AS rnk
        FROM electric_vehicle_sales_by_makers
        JOIN dim_date USING(date)
        WHERE
            fiscal_year = p_fiscal_year
            AND vehicle_category = p_category
        GROUP BY fiscal_year, maker
    ) ranked_data
    WHERE rnk <= p_n
    ORDER BY rnk;
END
```

```sql
#topmakers rnk stored procedure
call ev_db.get_ev_makersrnk(2023, '2-WHEELERS', 'top', 3);
call ev_db.get_ev_makersrnk(2023, '2-WHEELERS', 'bottom', 3);
call ev_db.get_ev_makersrnk(2024, '2-WHEELERS', 'top', 3);
call ev_db.get_ev_makersrnk(2024, '2-WHEELERS', 'bottom', 3);
```



fiscal_year	maker	total_ev_sold	rnk
2023	OLA ELECTRIC	152583	1
2023	OKINAWA	96945	2
2023	HERO ELECTRIC	88993	3

fiscal_year	maker	total_ev_sold	rnk
2023	JITENDRA	8563	1
2023	BEING	11018	2
2023	PURE EV	11556	3

fiscal_year	maker	total_ev_sold	rnk
2024	OLA ELECTRIC	322489	1
2024	TVS	180743	2
2024	ATHER	107552	3

fiscal_year	maker	total_ev_sold	rnk
2024	BATTRE ELECTRIC	4841	1
2024	REVOLT	7254	2
2024	KINETIC GREEN	9585	3

Q2 Top 5 States — EV Penetration Rate FY2024 (2W & 4W)

Objective

Identify which states have the highest EV adoption as a percentage of total vehicles sold. Analysed separately for 2-Wheelers and 4-Wheelers.

SQL Concepts Used

- SUM + GROUP BY
- Penetration Rate Formula
- ORDER BY DESC
- LIMIT
- JOIN + USING

Output Columns

- state
- penetration_rate (%)

SQL

```sql
#top5 state with highest ev penetration rate in fy 2024
select
state,
sum(electric_vehicles_sold) as total_EV_sold,
Sum(total_vehicles_sold)as Total_vehicle_sold,
round(
(sum(electric_vehicles_sold))/(sum(total_vehicles_sold))*100,2) as penetration_rate
from electric_vehicle_sales_by_state
join dim_date
using(date)
where fiscal_year=2024
group by state
order by penetration_rate desc
limit 5
```

100% 7:2

Result Grid Filter Rows: Search Export: Fetch rows:

state	total_EV_sold	Total_vehicle_sold	penetration_rate
Goa	10799	78524	13.75
Kerala	73938	638114	11.59
Karnataka	160989	1581988	10.18
Maharashtra	197169	2293994	8.60
Delhi	46724	606348	7.71

Q3 States with Negative Penetration Change (2022 vs 2024)

Objective

Detect states where EV adoption rate declined from FY2022 to FY2024. Identifies markets with resistance or challenges to EV adoption.

SQL Concepts Used

- CTE (WITH clause)
- Self JOIN
- Penetration change calc
- WHERE filter on delta

Output Columns

- state
- penetration_2022 (%)
- penetration_2024 (%)
- penetration_change (%)

SQL

```sql
#the negative p-rate change 2022 vs 2024
SELECT
    state,
    ROUND(SUM(CASE WHEN fiscal_year = 2022
        THEN electric_vehicles_sold END) /
        SUM(CASE WHEN fiscal_year = 2022
        THEN total_vehicles_sold END) * 100, 2) AS penetration_2022,

    ROUND(SUM(CASE WHEN fiscal_year = 2024
        THEN electric_vehicles_sold END) /
        SUM(CASE WHEN fiscal_year = 2024
        THEN total_vehicles_sold END) * 100, 2) AS penetration_2024,

    ROUND(SUM(CASE WHEN fiscal_year = 2024
        THEN electric_vehicles_sold END) /
        SUM(CASE WHEN fiscal_year = 2024
        THEN total_vehicles_sold END) * 100, 2)
    -
    ROUND(SUM(CASE WHEN fiscal_year = 2022
        THEN electric_vehicles_sold END) /
        SUM(CASE WHEN fiscal_year = 2022
        THEN total_vehicles_sold END) * 100, 2) AS penetration_change

FROM electric_vehicle_sales_by_state
```

50% 49:17

Result Grid Filter Rows: Search Export:

state	penetration_2022	penetration_2024	penetration_change
Sikkim	0.00	0.00	0.00
Nagaland	0.01	0.05	0.04
Andaman & Nic...	0.43	0.49	0.06
Arunachal Prad...	0.00	0.11	0.11
Meghalaya	0.02	0.36	0.34
Assam	0.19	0.64	0.45
Himachal Pradesh	0.45	0.90	0.45

Q4 Quarterly Trends — Top 5 EV Makers (4-Wheelers, 2022–2024)

Objective

Track quarter-by-quarter sales volume for the top 5 four-wheeler EV makers. Reveals growth trends and seasonal patterns per manufacturer.

SQL Concepts Used

- CTE (WITH clause)
- Subquery IN filter
- GROUP BY multi-column
- ORDER BY multi-column

Output Columns

- maker
- fiscal_year
- quarter (Q1–Q4)
- total_ev_sold

SQL

```sql
# top 5 makers
WITH top5_makers AS (
    SELECT maker
    FROM electric_vehicle_sales_by_makers
    JOIN dim_date USING(date)
    WHERE vehicle_category = '4-Wheelers'
    GROUP BY maker
    ORDER BY SUM(electric_vehicles_sold) DESC
    LIMIT 5
)
#Get quarterly trends for those makers
SELECT
    maker,
    fiscal_year,
    quarter,
    SUM(electric_vehicles_sold) AS total_ev_sold
FROM electric_vehicle_sales_by_makers
JOIN dim_date USING(date)
WHERE vehicle_category = '4-Wheelers'
AND maker IN (SELECT maker FROM top5_makers)
GROUP BY maker, fiscal_year, quarter
ORDER BY maker, fiscal_year, quarter;
```

23:19

ult Grid Filter Rows: Search Export:

maker	fiscal_year	quarter	total_ev_sold
BYD India	2022	Q1	0
BYD India	2022	Q2	0
BYD India	2022	Q3	1
BYD India	2022	Q4	32
BYD India	2023	Q1	81

Q5 Delhi vs Karnataka — EV Sales & Penetration Rate FY2024

Objective

Direct comparison of two key markets. Karnataka vs Delhi in terms of both absolute EV sales volume and penetration rate for 2W & 4W.

SQL Concepts Used

- IN filter for states
- Multiple aggregations
- GROUP BY state + category
- Penetration formula

Output Columns

- state
- vehicle_category
- total_ev_sold
- total_vehicles
- penetration_rate (%)

SQL

```sql
#delhi vs KA ev sale and penetration rate
select
    state,
    vehicle_category,
    sum(electric_vehicles_sold)as Total_ev_sold,
    sum(total_vehicles_sold)as total_veh_sold,
    round((sum(electric_vehicles_sold)/sum(total_vehicles_sold))*100,2) as penetration_rates
 from electric_vehicle_sales_by_state
 join dim_date
 using(date)
where state in ('Delhi','karnataka')
and fiscal_year = 2024
group by state,vehicle_category
ORDER BY state, vehicle_category;
```

100% 3:9

Result Grid Filter Rows: Search Export:

state	vehicle_category	Total_ev_sold	total_veh_sold	penetration_rates
Delhi	2-Wheelers	38094	405218	9.40
Delhi	4-Wheelers	8630	201130	4.29
Karnataka	2-Wheelers	148111	1279767	11.57
Karnataka	4-Wheelers	12878	302221	4.26

Q6 CAGR — Top 5 EV Makers (4-Wheelers, 2022–2024)

🎯 Objective

Calculate Compound Annual Growth Rate for top 5 four-wheeler makers. Uses custom CAGR function for clean, reusable calculation across queries.

💡 SQL Concepts Used

- User-Defined Function
- POWER() function
- CTE + CASE WHEN pivot
- CAGR = (end/start)^(1/n)-1

📊 Output Columns

- maker
- sales_2022
- sales_2024
- cagr_percent (%)

SQL

```sql
#qn6 cagr% 4w top5 makers from 2022 to 2024
WITH pivot_sales AS (
    SELECT
        maker,
        SUM(CASE WHEN fiscal_year = 2022 THEN electric_vehicles_sold END) AS sales_2022,
        SUM(CASE WHEN fiscal_year = 2024 THEN electric_vehicles_sold END) AS sales_2024
    FROM electric_vehicle_sales_by_makers
    JOIN dim_date USING(date)
    WHERE vehicle_category = '4-Wheelers'
    GROUP BY maker
    ORDER BY SUM(electric_vehicles_sold) DESC
    LIMIT 5
)
SELECT
    maker,
    sales_2022,
    sales_2024,
    calculate_cagr(sales_2022, sales_2024, 2) AS cagr_percent
FROM pivot_sales
ORDER BY cagr_percent DESC;
```

50% 19:10

Result Grid Filter Rows: Search Export:

maker	sales_2022	sales_2024	cagr_percent
BYD India	33	1466	566.52
Hyundai Motor	110	1390	255.48
Mahindra & Mahindra	4042	23346	140.33
MG Motor	1647	8829	131.53
Tata Motors	12708	48181	94.71

Q7 Top 10 States by CAGR — Total EV Sales (2022–2024)

Objective

Find states with the fastest growing EV markets using CAGR. Helps identify emerging high-growth markets for AtliQ's expansion strategy.

SQL Concepts Used

- User-Defined Function (CAGR)
- CTE pivot with CASE WHEN
- WHERE ev_2022 > 0 guard
- ORDER + LIMIT 10

Output Columns

- state
- ev_2022
- ev_2024
- cagr_percent (%)

SQL

```sql
WITH pivot_sales AS (
    SELECT
        state,
        SUM(CASE WHEN fiscal_year = 2022 THEN electric_vehicles_sold END) AS ev_2022,
        SUM(CASE WHEN fiscal_year = 2024 THEN electric_vehicles_sold END) AS ev_2024
    FROM electric_vehicle_sales_by_state
    JOIN dim_date USING(date)
    WHERE fiscal_year IN (2022, 2024)
    GROUP BY state
)
SELECT
    state,
    ev_2022,
    ev_2024,
    calculate_cagr(ev_2022, ev_2024, 2) AS cagr_percent
FROM pivot_sales
WHERE ev_2022 > 0
ORDER BY cagr_percent DESC
LIMIT 10;
```

50% 13:14

Result Grid Filter Rows: Search Export:

state	ev_2022	ev_2024	cagr_percent
Meghalaya	4	133	476.63
Tripura	28	304	229.50
Nagaland	1	9	200.00
Chandigarh	411	2877	164.58
Chhattisgarh	4534	28540	150.89
West Bengal	2685	16864	150.62
Goa	1778	10799	146.45
DNH and DD	35	198	137.85
Uttar Pradesh	10222	57758	137.70
Madhya Pradesh	7916	43223	133.67

Q8 Peak & Low Season Months for EV Sales (2022–2024)

Objective

Identify which months consistently see highest and lowest EV sales. Critical for launch timing, inventory planning and marketing campaigns.

SQL Concepts Used

- STR_TO_DATE() conversion
- MONTHNAME() function
- MONTH() for sort order
- Aggregate + ORDER BY

Output Columns

- month_name
- month_number
- total_ev_sold

SQL

```sql
SELECT
    YEAR(STR_TO_DATE(date, '%d-%b-%y')) AS year,
    MONTH(STR_TO_DATE(date, '%d-%b-%y')) AS month_number,
    MONTHNAME(STR_TO_DATE(date, '%d-%b-%y')) AS month_name,
    SUM(electric_vehicles_sold) AS total_ev_sold
FROM electric_vehicle_sales_by_state
WHERE YEAR(STR_TO_DATE(date, '%d-%b-%y')) IN (2022, 2023, 2024)
GROUP BY year, month_number, month_name
ORDER BY year, month_number;
```

100% 29:9

Result Grid Filter Rows: Search Export:

year	month_numb...	month_name	total_ev_sold
2022	1	January	31672
2022	2	February	38171
2022	3	March	58118
2022	4	April	55524
2022	5	May	45373
2022	6	June	47591
2022	7	July	50010
2022	8	August	55584
2022	9	September	56828
2022	10	October	81162
2022	11	November	80752
2022	12	December	68475
2023	1	January	68116
2023	2	February	70827

Q9 Projected EV Sales 2030 — Top 10 States by Penetration

🎯 Objective

Use CAGR to project 2030 EV sales for top 10 states. Gives AtliQ Motors a 6-year forward view for market entry and capacity planning.

💡 SQL Concepts Used

- CAGR Function reuse
- Multi-CTE chaining
- POWER() projection formula
- JOIN CTEs together

📊 Output Columns

- state
- ev_2024
- cagr_percent (%)
- projected_2030

SQL

```sql
WITH cte1 AS (
    SELECT
        state,
        SUM(CASE WHEN fiscal_year = 2022 THEN electric_vehicles_sold END) AS ev_2022,
        SUM(CASE WHEN fiscal_year = 2024 THEN electric_vehicles_sold END) AS ev_2024
    FROM electric_vehicle_sales_by_state
    JOIN dim_date USING(date)
    WHERE fiscal_year IN (2022, 2024)
    GROUP BY state
),
top10 AS (
    SELECT state
    FROM electric_vehicle_sales_by_state
    JOIN dim_date USING(date)
    WHERE fiscal_year = 2024
    GROUP BY state
    ORDER BY SUM(electric_vehicles_sold) / SUM(total_vehicles_sold) DESC
    LIMIT 10
)
SELECT
    p.state,
    p.ev_2024,
    calculate_cagr(p.ev_2022, p.ev_2024, 2) AS cagr_percent,
    ROUND(p.ev_2024 * POWER(1 + (calculate_cagr(p.ev_2022, p.ev_2024, 2) / 100), 6), 0) AS projected_2030
FROM cte1 p
JOIN top10 t ON p.state = t.state
WHERE p.ev_2022 > 0
ORDER BY projected_2030 DESC;
```

state	ev_2024	cagr_percent	projected_2030
Maharashtra	197169	101.89	13351421
Kerala	73938	132.83	11778808
Karnataka	160989	93.24	8382593
Chhattisgarh	28540	150.89	7117936
Odisha	39118	102.94	2732641
Goa	10799	146.45	2419672
Tamil Nadu	94314	59.95	1579362
Delhi	46724	68.10	1054257
Chandigarh	2877	164.58	986920
Puducherry	3098	105.44	232912

Q10 Revenue Growth Rate — 2W & 4W EVs (2022 vs 2024, 2023 vs 2024)

Objective

Estimate revenue using assumed average unit prices. Calculates growth rate for both vehicle segments across fiscal years.

SQL Concepts Used

- CASE WHEN for revenue calc
- Multi-year comparison
- Growth rate formula
- GROUP BY category

Output Columns

- vehicle_category
- units_2022
- units_2024
- growth_22_vs_24 (%)
- growth_23_vs_24 (%)

SQL

```sql
#Assumed avg prices: 4-Wheeler = 15,00,000 | 2-Wheeler = 85,000
SELECT
    vehicle_category,
    SUM(CASE WHEN fiscal_year = 2022 THEN electric_vehicles_sold END) AS units_2022,
    SUM(CASE WHEN fiscal_year = 2023 THEN electric_vehicles_sold END) AS units_2023,
    SUM(CASE WHEN fiscal_year = 2024 THEN electric_vehicles_sold END) AS units_2024,

   #calculate Revenue
    SUM(CASE WHEN fiscal_year = 2022 AND vehicle_category = '4-Wheelers'
        THEN electric_vehicles_sold * 1500000
        WHEN fiscal_year = 2022 AND vehicle_category = '2-Wheelers'
        THEN electric_vehicles_sold * 85000 END) AS revenue_2022,

    SUM(CASE WHEN fiscal_year = 2024 AND vehicle_category = '4-Wheelers'
        THEN electric_vehicles_sold * 1500000
        WHEN fiscal_year = 2024 AND vehicle_category = '2-Wheelers'
        THEN electric_vehicles_sold * 85000 END) AS revenue_2024,

     SUM(CASE WHEN fiscal_year = 2023 AND vehicle_category = '4-Wheelers'
        THEN electric_vehicles_sold * 1500000
```

61:32

Result Grid Filter Rows: Search Export:

vehicle_category	units_2022	units_2023	units_2024	revenue_2022	revenue_2024	revenue_2023	growth_rate_22_vs_24	growth_rate_23_vs_24
2-Wheelers	252573	727903	932692	21468705000	79278820000	61871755000	269.28	28.13
4-Wheelers	18577	47465	86901	27865500000	130351500000	71197500000	367.79	83.08

SQL Concepts Summary

Everything I used across all 10 questions

JOINs

JOIN + USING(date) to link tables

CTEs

WITH clause for clean query structure

Window Fn

RANK() OVER(PARTITION BY)

CASE WHEN

Conditional logic & pivot tables

Functions

calculate_cagr() reusable UDF

Procedures

get_ev_makers() with parameters